Ad Soyad: _____	**Tarih:** _____
Lab 03 – Hafta 3 – *Modern C++ ve STL Uygulamaları*	

Lab Hakkında

Bu laboratuvar çalışmasında C++20 ranges/views, string_view, std::format, std::optional, std::variant ve istisna yönetimini uygulamalı olarak kullanacaksınız. Her çalışma bölümünde önce örnek kodu inceleyip çalıştırın, ardından görevleri tamamlayın.

Süre: 2 saat **Derleme:** `g++ -std=c++20 -o program dosya.cpp`

1 Çalışma 1: Ranges ve Projeksiyonlar (25 dk)

Ön Bilgi

C++20 ile gelen `std::ranges` kütüphanesi, algoritmalara doğrudan kapsayıcı geçirmenizi sağlar – artık `begin()/end()` çağrısına gerek yoktur. **Projeksiyonlar** ise bir elemanın hangi üyesine göre işlem yapılacağını belirtir. **Views**, verileri kopyalamadan tembel (lazy) dönüşüm zincirleri oluşturmanızı sağlar.

Aşağıdaki başlangıç kodunu `calisma1.cpp` dosyasına yazın ve derleyin:

```cpp
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>
#include <ranges>

struct Ogrenci {
    int numara;
    std::string isim;
    double gpa;
};

int main() {
    std::vector<Ogrenci> liste = {
        {2001, "Ali Yilmaz",    3.45},
        {2002, "Ayse Kaya",     3.80},
        {2003, "Mehmet Demir",  2.30},
        {2004, "Zeynep Can",    3.60},
        {2005, "Can Ozturk",    2.10},
        {2006, "Elif Aksoy",    3.95},
        {2007, "Burak Celik",   2.75},
        {2008, "Deniz Yildiz",  3.20}
    };

    // ranges::sort ile GPA'ya gore sirala (buyukten kucuge)
    std::ranges::sort(liste, std::greater{}, &Ogrenci::gpa);

    std::cout << "--- GPA Siralamasi ---\n";
    for (const auto& o : liste) {
        std::cout << o.isim << " - GPA: " << o.gpa << '\n';
    }

    // ranges::find_if ile ilk dusuk GPA'li ogrenciyi bul
    auto it = std::ranges::find_if(liste,
        [](double g) { return g < 2.5; }, &Ogrenci::gpa);
```

```

    if (it != liste.end()) {
        std::cout << "\nIlk dusuk GPA: " << it->isim
                  << " (" << it->gpa << ")\n";
    }

    return 0;
}
```

Pipe Operatörü ve Projeksiyonlar

C++20 views pipe (`|`) operatörü ile zincirlenir:
`liste | views::filter(kosul) | views::transform(donusum)`
Projeksiyonlar algoritmalara ek parametre olarak geçilir:
`ranges::sort(liste, std::less{}, &Ogrenci::isim)`
Bu sayede karşılaştırma fonksiyonu yazmadan doğrudan üye değişkene göre sıralama yapabilirsiniz.

Görev 1.1 – ranges::sort ile Projeksiyon (5 dk)

Öğrenci listesini **isme göre** alfabetik sıralayın. Özel bir karşılaştırma fonksiyonu yazmak yerine **projeksiyon** kullanın:

```
std::ranges::sort(liste, /* karsilastirma */, /* projeksiyon */);
```

a) `std::less{}` karşılaştırıcısı ve `&Ogrenci::isim` projeksiyonu ile A-Z sıralama yapın.

b) Sıralanmış listeyi yazdırın.

Görev 1.2 – views::filter ve views::transform Pipeline (10 dk)

GPA ortalaması 2.5'in üzerinde olan öğrencilerin isimlerini büyük harfe çeviren bir pipeline oluşturun:

a) `views::filter` ile GPA > 2.5 koşulunu uygulayın.

b) `views::transform` ile her öğrencinin ismini büyük harfe çevirin. Bunun için bir yardımcı lambda yazın:

```
auto buyuk_harf = [](std::string s) {
    for (auto& c : s) c = std::toupper(c);
    return s;
};
```

c) Pipeline'ı pipe (`|`) operatörü ile oluşturun ve sonuçları yazdırın.

Görev 1.3 – views::take ve views::drop (10 dk)

a) Listeyi GPA'ya göre büyükten küçüğe sıralayın.

b) `views::take(3)` ile en yüksek GPA'lı **ilk 3** öğrenciyi yazdırın.

c) `views::drop(liste.size() - 2)` ile en düşük GPA'lı **son 2** öğrenciyi yazdırın.

d) Her iki sonucu da `std::ranges::for_each` ile yazdırın.

Dikkat: Views Tembel Değerlendirilir

Views zincirleri tanımlandıklarında çalışmaz; yalnızca üzerlerinde iterasyon yapıldığında (for döngüsü, `ranges::for_each`, vb.) değerlendirilir. Bu nedenle bir view'ı bir değişkene atayabilir ve sonra birden fazla kez kullanabilirsiniz – ancak kaynak veri değişmişse sonuçlar da değişir.

2 Çalışma 2: string_view ve std::format (25 dk)

Ön Bilgi

`std::string_view` (C++17) bir karakter dizisine salt-okunur, kopyalamayan bir referanstır. Fonksiyon parametrelerinde `const std::string&` yerine kullanılarak gereksiz kopyalama önlenir. `std::format` (C++20) ise Python'daki `f-string` benzeri tip-güvenli biçimlendirme sağlar.
Yaşam süresi tuzağı: `string_view` sahiplik almaz; işaret ettiği string yok edilirse dangling reference oluşur!

Aşağıdaki kodu `calisma2.cpp` dosyasına yazın:

```cpp
#include <iostream>
#include <string>
#include <string_view>
#include <format>
#include <vector>

// string_view alan fonksiyon -- kopyalama yapmaz
void bilgi_yazdir(std::string_view baslik, std::string_view icerik) {
    std::cout << std::format("[{}] {}\n", baslik, icerik);
}

struct Ogrenci {
    int numara;
    std::string isim;
    double gpa;
};

int main() {
    // string_view temel kullanim
    std::string ad = "Ali Yilmaz";
    std::string_view sv = ad; // kopyalamaz, gosterir
    bilgi_yazdir("Ogrenci", sv);

    // std::format ile formatlama
    std::vector<Ogrenci> liste = {
        {2001, "Ali Yilmaz",    3.45},
        {2002, "Ayse Kaya",     3.80},
        {2003, "Mehmet Demir",  2.30}
    };

    // Tablo basligi
    std::cout << std::format("\n{:<6} {:<20} {:>6}\n",
                             "No", "Isim", "GPA");
    std::cout << std::string(34, '-') << '\n';

    for (const auto& o : liste) {
        std::cout << std::format("{:<6} {:<20} {:>6.2f}\n",
                                 o.numara, o.isim, o.gpa);
    }

    return 0;
}
```

`std::format` Belirteçleri

Belirteç	Açıklama
`{:<20}`	20 karakter, sola hizalı
`{:>10}`	10 karakter, sağa hizalı
`{:^15}`	15 karakter, ortaya hizalı
`{:06}`	6 basamak, sıfır dolgulu
`{:.2f}`	2 ondalık basamak, kayan nokta
`{:08.2f}`	8 karakter, sıfır dolgulu, 2 ondalık

Görev 2.1 – string_view ile Doğrulama Fonksiyonu (10 dk)

`std::string_view` parametresi alan bir `eposta_gecerli_mi` fonksiyonu yazın:

a) Fonksiyon, gelen string_view'ın `@` karakteri içerip içermediğini `find` ile kontrol etsin.

b) `starts_with("ogrenci.")` ile öğrenci e-postası olup olmadığını kontrol etsin (C++20).

c) `ends_with(".edu.tr")` ile geçerli alan adını doğrulasın (C++20).

d) Fonksiyonu en az 3 farklı e-posta ile test edin.

```
bool eposta_gecerli_mi(std::string_view eposta) {
    // Buraya kodunuzu yazin
}
```

Görev 2.2 – std::format ile Tablo Oluşturma (10 dk)

Öğrenci bilgilerini hizalı bir tablo olarak yazdırın:

a) Tablo sütunları: Numara (6 hane, sıfır dolgulu), İsim (25 karakter, sola hizalı), GPA (6 karakter, 2 ondalık, sağa hizalı), Durum (10 karakter, ortaya hizalı).

b) GPA $\geq$ 2.0 ise durum "Basarili", değilse "Kaldi" olsun.

c) Tablo başlığı ve ayırıcı çizgi ekleyin.

Beklenen çıktı formatı:

```
 No      Isim                        GPA     Durum
-----------------------------------------------------
 002001  Ali Yilmaz                   3.45   Basarili
 002003  Mehmet Demir                 2.30   Basarili
```

Görev 2.3 – Dangling Reference Düzeltme (5 dk)

Aşağıdaki kodda bir **yaşam süresi hatası** (dangling reference) vardır. Hatayı bulun ve düzeltin:

```
std::string_view tehlikeli_fonksiyon() {
    std::string gecici = "Merhaba Dunya";
    return gecici; // [!] Tehlikeli! Neden?
}

int main() {
    auto sv = tehlikeli_fonksiyon();
    std::cout << sv << '\n'; // Tanimsiz davranis!
}
```

a) Hatanın nedenini bir yorum satırı ile açıklayın.

b) İki farklı düzeltme yöntemi önerin ve uygulayın.

Dikkat: string_view Yaşam Süresi

`string_view`, kendisine atanan string'in sahipliğini almaz. Geçici bir `std::string` döndüren fonksiyondan `string_view` döndürmek **tanımsız davranışa** yol açar. Fonksiyon dönüş değeri olarak her zaman `std::string` kullanın.

3 Çalışma 3: optional ve variant (25 dk)

Ön Bilgi

`std::optional<T>` (C++17) bir değerin var olup olmadığını güvenle ifade eder – `nullptr` veya sihirli değerler (-1, 0) kullanmak yerine. `std::variant<T1, T2, ...>` ise birden fazla türden tam olarak birini tutabilen tip-güvenli bir birleşimdir (union). `std::visit` ile variant içindeki değere polimorfik erişim sağlanır.

Aşağıdaki kodu `calisma3.cpp` dosyasına yazın:

```cpp
#include <iostream>
#include <string>
#include <optional>
#include <variant>
#include <vector>
#include <map>

// optional donduren arama fonksiyonu
std::optional<std::string> ogrenci_bul(
    const std::map<int, std::string>& kayitlar, int numara)
{
    auto it = kayitlar.find(numara);
    if (it != kayitlar.end()) {
        return it->second; // deger var
    }
    return std::nullopt; // deger yok
}

int main() {
    std::map<int, std::string> kayitlar = {
        {2001, "Ali Yilmaz"},
        {2002, "Ayse Kaya"},
        {2003, "Mehmet Demir"}
    };

    // optional kullanimi
    auto sonuc = ogrenci_bul(kayitlar, 2002);
    if (sonuc.has_value()) {
        std::cout << "Bulundu: " << sonuc.value() << '\n';
    }

    // value_or ile varsayilan deger
    auto sonuc2 = ogrenci_bul(kayitlar, 9999);
    std::cout << "Arama: "
              << sonuc2.value_or("Bulunamadi") << '\n';

    // variant: Not turlerini temsil etme
    using Not = std::variant<int, double, std::string>;

    std::vector<Not> notlar;
    notlar.push_back(85);              // sayi notu
    notlar.push_back(3.45);            // GPA
    notlar.push_back(std::string("Devamsiz")); // metin

    for (const auto& n : notlar) {
        if (std::holds_alternative<int>(n)) {
            std::cout << "Sayi notu: " << std::get<int>(n) << '\n';
```

```
        } else if (std::holds_alternative<double>(n)) {
            std::cout << "GPA: " << std::get<double>(n) << '\n';
        } else {
            std::cout << "Durum: " << std::get<std::string>(n)
                      << '\n';
        }
    }

    return 0;
}
```

`value_or` ve Overloaded Deseni

`optional::value_or(varsayilan)` değer yoksa varsayılanı döner.
Variant için **overloaded** deseni, `std::visit` ile tüm türleri tek bir yerde ele almanızı sağlar:

```
template<class... Ts>
struct overloaded : Ts... { using Ts::operator()...; };

std::visit(overloaded{
    [](int n) { /* sayi islemi */ },
    [](double d) { /* GPA islemi */ },
    [](const std::string& s) { /* metin islemi */ }
}, not_degeri);
```

Görev 3.1 – optional ile Güvenli Arama (10 dk)

Öğrenci GPA'sını arayan bir fonksiyon yazın:

a) `std::optional<double>` döndüren `gpa_bul` fonksiyonunu yazın. Parametre: öğrenci vektörü ve numara.

b) Öğrenci bulunursa GPA'sını, bulunamazsa `std::nullopt` döndürün.

c) Çağrı tarafında `has_value()` kontrolü yapın ve bulunamayan durumda `value_or(0.0)` ile varsayılan değer kullanın.

d) 3 farklı numara ile test edin (2 mevcut, 1 mevcut olmayan).

```
std::optional<double> gpa_bul(
    const std::vector<Ogrenci>& liste, int numara) {
    // Buraya kodunuzu yazin
}
```

Görev 3.2 – variant ile Not Sistemi (10 dk)

Bir not sistemi tasarlayın:

a) `using Not = std::variant<char, double, std::string>;` tipini tanımlayın. `char` harf notu ('A', 'B', ...), `double` sayısal not (0.0–4.0), `std::string` durum bilgisi ("Devamsiz", "Muaf") temsil eder.

b) 5 farklı not değeri oluşturun (en az bir harf, bir sayısal, bir string).

c) `std::holds_alternative` ile her notun türünü kontrol edip uygun şekilde yazdırın.

Görev 3.3 – std::visit ile Overloaded Deseni (5 dk)

Görev 3.2'deki notları `std::visit` ve **overloaded** deseni ile işleyin:

a) Aşağıdaki `overloaded` yardımcı yapıyı kodunuza ekleyin:

```
template<class... Ts>
struct overloaded : Ts... { using Ts::operator()...; };
```

b) `std::visit` ile her not türü için farklı bir çıktı üretin: harf notu → "Harf: X", sayısal → "GPA: X.XX", durum → "Durum: ..."

c) Tüm notlar üzerinde bir for döngüsü ile visit çağırın.

Dikkat: std::get Güvenliği

`std::get<T>(v)` ile yanlış tür istenirse `std::bad_variant_access` istisnası fırlatılır. Güvenli erişim için önce `holds_alternative<T>(v)` kontrolü yapın veya `std::visit` kullanın.

4 Çalışma 4: OBYS Entegrasyonu (25 dk)

Ön Bilgi

Bu bölümde önceki üç çalışmada öğrendiğiniz ranges, string_view, format, optional ve variant kavramlarını OBYS (Öğrenci Bilgi Yönetim Sistemi) projesinde birleştireceğiz. Amaç, modern C++20 araçlarını gerçek bir proje bağlamında kullanmaktır.

Aşağıdaki OBYS iskelet kodunu `calisma4.cpp` dosyasına yazın:

```cpp
#include <iostream>
#include <string>
#include <string_view>
#include <format>
#include <vector>
#include <map>
#include <ranges>
#include <algorithm>
#include <optional>
#include <variant>

struct Ogrenci {
    int numara;
    std::string isim;
    std::string bolum;
    double gpa;
};

// Not turu: harf, sayi veya ozel durum
using Not = std::variant<char, double, std::string>;

// Overloaded yardimcisi
template<class... Ts>
struct overloaded : Ts... { using Ts::operator()...; };

class OBYS {
    std::vector<Ogrenci> ogrenciler_;

public:
    void ekle(Ogrenci o) {
        ogrenciler_.push_back(std::move(o));
    }

    // Tum ogrencileri dondur (const referans)
    const std::vector<Ogrenci>& tumu() const {
        return ogrenciler_;
    }

    // Numara ile arama (optional)
    std::optional<Ogrenci> bul(int numara) const {
        auto it = std::ranges::find(ogrenciler_,
                                    numara, &Ogrenci::numara);
        if (it != ogrenciler_.end()) return *it;
        return std::nullopt;
    }
};

int main() {
```

```
    OBYS sistem;
    sistem.ekle({2001, "Ali Yilmaz",     "Bilgisayar", 3.45});
    sistem.ekle({2002, "Ayse Kaya",      "Bilgisayar", 3.80});
    sistem.ekle({2003, "Mehmet Demir",   "Elektrik",   2.30});
    sistem.ekle({2004, "Zeynep Can",     "Bilgisayar", 3.60});
    sistem.ekle({2005, "Can Ozturk",     "Makine",     2.10});
    sistem.ekle({2006, "Elif Aksoy",     "Bilgisayar", 3.95});
    sistem.ekle({2007, "Burak Celik",    "Elektrik",   2.75});
    sistem.ekle({2008, "Deniz Yildiz",   "Bilgisayar", 3.20});

    // Baslangic: Tum ogrencileri yazdir
    std::cout << "--- Tum Ogrenciler ---\n";
    for (const auto& o : sistem.tumu()) {
        std::cout << std::format("{} - {} ({})\n",
                                  o.numara, o.isim, o.gpa);
    }

    return 0;
}
```

Görev 4.1 – Ranges Pipeline ile Sorgulama (10 dk)

`main()` fonksiyonunda aşağıdaki sorgulamayı ranges pipeline ile gerçekleştirin:

a) "Bilgisayar" bölümündeki öğrencileri `views::filter` ile filtreleyin.

b) Filtrelenen öğrencileri GPA'ya göre sıralayın (`ranges::sort` projeksiyonla).

c) `views::transform` ile her öğrencinin bilgisini `std::format` ile biçimlendirin.

d) Sonuçları `views::take(3)` ile ilk 3 öğrenci ile sınırlayın.

e) Sonucu ekrana yazdırın. Beklenen format:

```
--- Bilgisayar En Basarili 3 ---
 002006 Elif Aksoy            3.95
 002002 Ayse Kaya             3.80
 002004 Zeynep Can            3.60
```

Görev 4.2 – optional ile Arama + variant ile Notlar (10 dk)

a) `sistem.bul(2004)` ile Zeynep Can'ı arayın. Sonucu `optional` ile kontrol edin ve bilgilerini yazdırın.

b) `sistem.bul(9999)` ile mevcut olmayan bir öğrenciyi arayın. `value_or` benzeri bir yaklaşımla "Bulunamadi" mesajı yazdırın (optional'ın has_value kontrolü ile).

c) Her öğrenciye bir `Not` atayan bir `std::map<int, Not>` oluşturun. En az 3 farklı not türü kullanın:

```
std::map<int, Not> not_tablosu;
not_tablosu[2001] = 'A';
not_tablosu[2002] = 3.80;
not_tablosu[2003] = std::string("Devamsiz");
```

d) `std::visit` ve overloaded deseni ile not tablosunu yazdırın.

Bonus: Özel İstisna Sınıfı ve Hata Yönetimi (10 dk)

OBYS sınıfına istisna tabanlı hata yönetimi ekleyin:

a) `std::runtime_error`'dan türeyen bir `OgrenciBulunamadiHatasi` sınıfı yazın:

```
class OgrenciBulunamadiHatasi : public std::runtime_error {
    int numara_;
public:
    OgrenciBulunamadiHatasi(int no)
        : std::runtime_error(
            std::format("Ogrenci bulunamadi: {}", no)),
          numara_(no) {}
    int numara() const { return numara_; }
};
```

b) OBYS sınıfına `bul_veya_hata(int numara)` metodu ekleyin. Öğrenci bulunamazsa bu istisnayı fırlatsın.

c) `try/catch` bloğu ile hem mevcut hem mevcut olmayan öğrenciyi arayın.

Teslim Bilgileri

Teslim edilecekler:

1. `calisma1.cpp` – Ranges ve projeksiyonlar (Görev 1.1–1.3)
2. `calisma2.cpp` – string_view ve format (Görev 2.1–2.3)
3. `calisma3.cpp` – optional ve variant (Görev 3.1–3.3)
4. `calisma4.cpp` – OBYS entegrasyonu (Görev 4.1–4.2)
5. *Bonus:* Özel istisna sınıfı (Bonus)

Puanlama: Çalışma 1: 25p Çalışma 2: 25p Çalışma 3: 25p Çalışma 4: 20p Bonus: 10p